

Cache Coherence Protocols on Sparse-Matrix Vector Multiplication (SpMV) and Graphs in GEM5

Kevin Le

Department of Electrical and Computer Engineering
University of California, Davis
Davis, CA 95616
kinle@ucdavis.edu

Vivian Ly Nguyen

Department of Electrical and Computer Engineering
University of California, Davis
Davis, CA 95616
vilnguyen@ucdavis.edu

Kilho Chang

Department of Electrical and Computer Engineering
University of California, Davis
Davis, CA 95616
khjchang@ucdavis.edu

Yaseen Alkhameri

Department of Electrical and Computer Engineering
University of California, Davis
Davis, CA 95616
yalkhameri@ucdavis.edu

Abstract—The rise in popularity of machine learning and its applicability across many domains has increased interest in supporting its unique workload demands. Two common and challenging workloads are sparse matrix-dense vector multiplication (SpMV) and graph algorithms, which are deeply embedded in the internal operations of many machine learning models. In this paper, we analyze how these workloads and their irregular memory-access patterns affect coherence traffic in multicore processors. Our results show that the MOESI directory protocol addresses some of the coherence challenges associated with these workloads, while the other studied protocols, MESI and MI, provide fewer states to simplify implementation and reduce expensive message traffic on the directory network. Motivated by the increase in Forward_GETX requests observed in our experiments, we propose and discuss the implications of a specialized “Delegated” state that allows a core to send atomic write instructions to another core holding the modified cache block, thereby reducing traffic over the interconnect.

I. INTRODUCTION

The unprecedented scaling of Machine Learning (ML) and Artificial Intelligence (AI) models has fundamentally transformed the computing landscape, driving the need for highly parallel, high-performance hardware architectures. While early Deep Learning successes were dominated by dense, regular computations—such as dense Matrix-Matrix Multiplication (GEMM) found in Convolutional Neural Networks (CNNs)—the frontier of ML has rapidly shifted toward highly sparse and unstructured domains. Today, Graph Neural Networks (GNNs), large-scale recommender systems, and sparse attention mechanisms in Large Language Models (LLMs) represent a growing fraction of datacenter workloads. At the algorithmic core of these emerging models lie Graph processing operations and Sparse Matrix-Vector Multiplication (SpMV).

Unlike their dense counterparts, SpMV and Graph workloads present a unique set of challenges for modern multicore processors. Standard ML hardware, optimized for dense linear algebra, relies heavily on predictable spatial locality

to maximize cache utility and memory bandwidth. However, graph data and sparse matrices are inherently irregular. They are characterized by data-dependent execution paths, exceptionally low compute-to-memory-access ratios, and highly unpredictable memory reference streams.

In SpMV operations, the non-zero elements of a sparse matrix are compressed into formats like Compressed Sparse Row (CSR) or Coordinate List (COO). Multiplying these structures by a dense input vector requires indirect memory addressing (e.g., `vector[indexArray[i]]`). In a parallel execution environment, this indirect addressing effectively shatters data locality. As cores fetch scattered elements from memory, caches are quickly polluted with entire cache lines where only a single word is useful, leading to severe bandwidth underutilization and soaring cache miss rates.

Compounding the problem of poor data locality is the profound stress these workloads place on multi-core cache coherency protocols. Modern processors utilize directory-based or snooping-based coherency protocols (such as MESI or MOESI) to ensure that all cores maintain a consistent view of shared memory. These protocols were historically designed under the assumption of spatial locality and predictable, coarse-grained data sharing. Under the chaotic sharing patterns of Graph and SpMV workloads, these assumptions break down entirely.

When multiple cores process adjacent nodes in a graph or overlapping rows in a sparse matrix, they frequently attempt to read and update shared data structures, such as vertex feature arrays or accumulation vectors. Because the data structures are laid out contiguously in memory but accessed seemingly at random, independent updates frequently fall onto the same physical cache line. This triggers massive amounts of false sharing.

Consequently, cache lines are subjected to intense “ping-ponging” between the private L1 and L2 caches of different cores. The coherence directory becomes overwhelmed by a

storm of invalidation messages and read-to-share requests. This excessive coherence traffic not only consumes vital on-chip network interconnect bandwidth but also stalls the processing cores, forcing them to wait hundreds of cycles to acquire write permissions for data they inherently share.

Despite the critical role of sparse operations in modern ML, the microarchitectural toll they extract via cache coherence protocols remains underexplored. Existing literature often focuses on memory bandwidth limitations or algorithmic load-balancing, treating the coherency subsystem as a transparent layer. However, as core counts continue to scale to meet the demands of massive ML models, the overhead of maintaining cache coherence in the presence of irregular sharing patterns threatens to become the primary bottleneck to parallel efficiency.

This paper aims to bridge that gap by rigorously quantifying and analyzing the impact of ML-driven SpMV and Graph workloads on cache coherency protocols. We systematically evaluate how varying degrees of matrix sparsity, graph algorithms, and core scaling influence coherency traffic, invalidation rates, and exclusive access requests. By exposing the microarchitectural inefficiencies of traditional coherency state machines under sparse execution, this work lays the initial experimental investigation for designing future domain-specific coherency optimizations and hardware accelerators tailored for the next generation of irregular Machine Learning.

II. RELATED WORK AND LIMITATIONS OF EXISTING WORK

This paper compares how directory-based cache coherence protocols affect coherence activity in sparse-matrix and graph workloads. We evaluate three directory-based coherence protocols in Gem5 Ruby: MI_Example, MESI_Two_Level, and MOESI_CMP_directory. Because all three protocols are evaluated within the same Ruby-based memory-system framework, the observed differences mainly reflect coherence protocol behavior.

A. MI_Example

MI_example is the simplest directory based cache coherency protocol. It contains two states: Modified (M) and Invalid (I). While in the Modified state, the cache block is in one cache, which holds the only valid copy of the data. While in the Invalid state, the cache does not hold a valid copy of the cache block. Since this protocol only has these two states, it cannot distinguish between shared data and data in which it is the sole owner. This causes any access to a block to require obtaining exclusive ownership from the directory if the block is not already in the Modified state. This results in both load and store operations that miss to generate a GETX request to the directory. If another cache already holds the block in the Modified state, the directory forwards the request to that cache using a FWD_GETX message, with the current owner transferring the cache block and invalidating its own copy. Since the protocol lacks additional states, ownership of a block moves frequently between caches when multiple

cores access the same data. As shown in Fig. 1, this behavior follows directly from the limited M and I state transitions. As a result, the MI protocol generates a large amount of coherence traffic, with many accesses requiring ownership transfers and full GETX transactions.

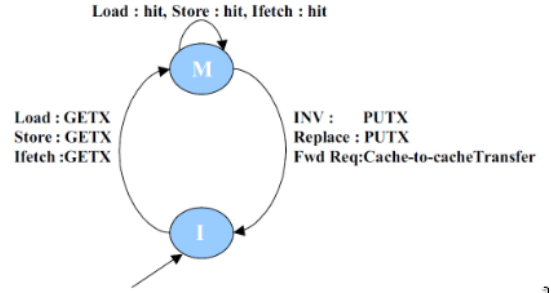


Fig. 1. A state diagram showing the states and transitions of the MI_Example implementation in Gem5. There are only two states in the protocol: Modified (M) and Invalid (I). The transition of M to I requires a PUTX request to write back the dirty data back to main memory. The transition of I to M requires a GETX request from the requesting processor, due to the lack of a shared state in the protocol.

B. MESI_Two_Level

As shown in Fig. 2, MESI_Two_Level extends the baseline two-state design by adding the Shared (S) and Exclusive (E) states within a two-level inclusive cache hierarchy. In this hierarchy, the directory is integrated with the shared L2 cache and tracks coherence state for each cache block across the private L1 caches. The Shared state allows multiple cores to hold clean copies of the same block, making read sharing more efficient. The Exclusive state is used when a block is held by only one core and has no other sharers. In this case, the block remains privately owned and can later be written without an additional coherence transaction, reducing unnecessary write-related traffic. In Gem5 Ruby, the protocol also supports silent eviction of clean L1 blocks, as a result, clean private cache lines can be removed without generating extra coherence traffic. Therefore, the benefit of MESI_Two_Level over MI_Example should not be attributed to the Exclusive state alone. Instead, it comes from the combined effect of efficient clean sharing through the Shared state, lower write-related overhead through the Exclusive state, and reduced control traffic through silent eviction [4].

C. MOESI_CMP_directory

As shown in Fig. 3, the MOESI (Modified-Owned-Exclusive-Shared-Invalid) protocol adds on the “Owned” state to the MESI_Two_Level protocol. When a block in a cache is in the modified state, a request from another cache to read the same block will issue a GETS (read request) will be sent to the directory. In the directory implementation, the directory will then search to see if there is a cache that has the block in either the modified or the owned state. If found, the directory issues a FWD_GETS to the cache that has the block, and it will forward the value to the requesting cache. This happens without issuing a writeback to main memory, so the directory


```

root@3991e1d70b14:/gem5/gem5# ./build/X86/gem5.opt \
--outdir=m5out_moesi_4cores_1000iterations \
configs/deprecated/example/se.py \
--ruby \
--num-cpus=4 \
--cpu-type=X86MinorCPU \
--cacheline_size=128 \
--l1_size=32kB --l1d_size=32kB \
--l1_assoc=1 --l1d_assoc=1 \
--l2cache --l2_size=512kB --l2_assoc=1 \
--cmd=$PWD/my_tests/matrix_test \
--options="$PWD/my_tests/hb685.mtx 1000 4"
gem5 Simulator System. https://www.gem5.org

```

Fig. 4. Running the X86 binary on the se (syscall emulator) configuration file on the multiplication workload, specifying the cache sizes, matrix files, and number of threads and CPUs to run using Gem5.

```

----- Begin Simulation Statistics -----
simSeconds          0.000000000000000000 # Number of seconds simulated (Second)
simTicks            39982216580 # Number of ticks simulated (Tick)
finalTick           39982216580 # Number of ticks from beginning of simulation (
restored from checkpoints and never reset) (Tick)
simFreq             1000000000000 # The number of ticks per simulated second ((Ti
of/Second))
hostSeconds          791.56 # Real time elapsed on the host (Second)
hostTickRate        39140897 # The number of ticks simulated per host second
((ticks/s)) ((Tick/Second))
hostMemory          824380 # Number of bytes of host memory used (Byte)
simInsts            110518056 # Number of instructions simulated (Count)
simOps              102457955 # Number of ops (including micro ops) simulated
((Count))
hostInstRate        139611 # Simulator instruction rate (Inst/s) ((Count/Se
cond))
hostOpRate          243138 # Simulator op (including micro ops) rate (op/s)
((Count/Second))
system_clk_domain_clock 1000 # Clock period in ticks (Tick)
system_cpu0_numcycles 61964913 # Number of cpu cycles simulated (Cycle)
system_cpu0_cpi      3.240170 # CPI: cycles per instruction (core level) ((Cyc
le/Count))
system_cpu0_ipc      0.308245 # IPC: instructions per cycle (core level) ((Cou

```

Fig. 5. A preview of a Gem5 stats.txt file, detailing the simulated time for running the program, simulated CPI, host instruction rate, memory used, etc. Coherence-related activity will be discussed later in Section V of this paper.

in and perform multiplication on, the second is the number of iterations to run (set to 1000 for all our experiments), and the third is the number of threads to create in software. This should match the number of CPU cores defined in order to create single-thread multicore behavior, and the command format used in our experiments is shown in Fig. 4.

Category	Parameter	Value
Processor	Core Count	2, 4, 8 Cores
	CPU Model	X86MinorCPU
Cache Hierarchy	Memory Model	Ruby Memory System
	Cache Line Size	128 Bytes
	Capacity (Each)	32 kB
L1 Cache (I/D)	Associativity	1 (Direct-mapped)
	Latency	2 Cycles
	Capacity	512 kB
L2 Cache	Associativity	1 (Direct-mapped)
	Latency	10 Cycles
Coherence	Protocols	MI_Example, MESI_Two_Level, MOESI_CMP_directory
Workloads	Benchmarks	GAPBS BFS (scale 12); SpMV (hb685, bp_400, bcsstm12); 1000 iter.

TABLE I: Simulation Configuration

The experiments we conducted ran the MI_Example, MESI_Two_Level, and MOESI_CMP_directory protocols. We ran the binaries with a 128-byte cache line, 32 kB L1 instruction and data caches, and a 512 kB L2 cache. All of these caches were direct-mapped to keep the cache organization uniform and to make coherence-related effects easier to observe. We also set the L1 cache latency to 2 cycles and the L2 cache latency to 10 cycles in the configuration files to model a realistic cache. Figure 4 details an example command in Gem5 to run the settings on matrix multiplication with a matrix with 4 cores and 1000 iterations. The workloads we ran were GAPBS (GAP Benchmark Suite [3]) Breadth-First Search (BFS) and sparse matrix-vector multiplication using our custom workload on the symmetric (HB685), asymmetric (BP400), and large symmetric (Bcsstm12) matrices to evaluate changes in directory messages across different matrix configurations. We recorded directory signals and forwarding metrics that reflect the behavior of each Gem5 state, including GETX, GETS, and FWD_GETS messages. We extracted the results from the stats.txt file, which is generated automatically after running the binary with the specified protocol, as illustrated in Fig. 5. The complete simulation parameters used in these experiments are summarized in Table I, and the selected sparse-matrix and graph workloads are summarized in Table II.

Matrix	Dim.	NNZ	Characteristic
HB/685_bus	685×685	3,249	Small, Symmetric
HB/bp_400	822×822	4,790	Medium, Unsymmetric
HB/bcsstm12	1473×1473	19,659	Large, Symmetric
GAPBS BFS	Scale 12	—	Graph traversal

TABLE II: Workloads

IV. RESULTS

A. MI_Example

1) *PUTX (Invalidation Proxy)*: PUTX events correspond to modified cache blocks being written back to the directory when they are evicted from a cache. In this study, PUTX serves as a proxy for invalidation-related behavior. Unlike GETX and FWD_GETX, PUTX activity does not consistently increase with core count; instead, it varies depending on the workload and how its working set interacts with the cache.

For the HB/685_bus workload, PUTX activity increases as the number of cores grows. As shown in Fig. 6, the number of PUTX messages rises from 371K at 2 cores to 378K at 4 cores and 477K at 8 cores. The HB/bcsstm12 workload behaves differently, showing a different trend across core counts. As shown in Fig. 7, PUTX messages change from 2.3M at 2 cores to 2.1M at 4 cores and 1.9M at 8 cores.

These results suggest that PUTX is influenced more by cache-eviction patterns than by coherence sharing alone. Further validation would be needed to determine whether the

observed PUTX behavior for bcsstm12 reflects a true workload effect or a case of measurement sensitivity.

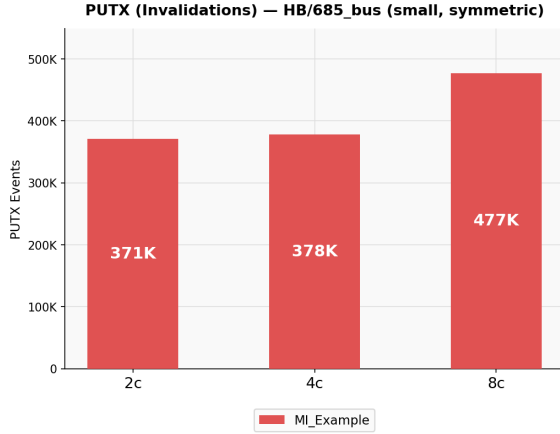


Fig. 6. PUTX activity for MI_Example on HB/685_bus across 2, 4, and 8 cores.

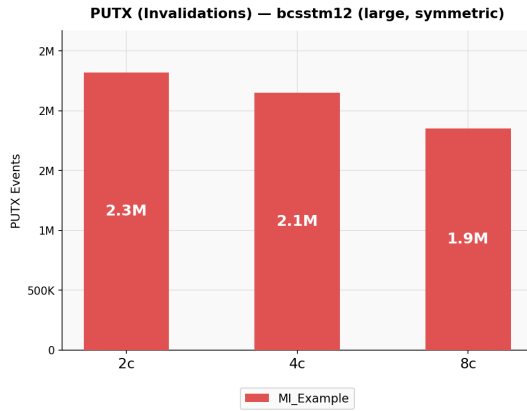


Fig. 7. PUTX activity for MI_Example on HB/bcsstm12 across 2, 4, and 8 cores.

2) *GETX*: GETX traffic represents ownership requests issued when a core needs write or read permission for a cache line. Since MI has only the Modified and Invalid states, it cannot distinguish between shared data and data for which it is the sole owner. As a result, any cache miss that requires access to a block must be resolved by obtaining exclusive ownership through a GETX message.

Across all workloads, GETX activity increases as the number of cores increases. For the HB/685_bus workload, as shown in Fig. 8, GETX requests increase from 5.85M at 2 cores to 12.69M at 4 cores and 17.88M at 8 cores. A similar trend appears in the HB/bcsstm12 workload, where, as shown in Fig. 9, GETX traffic rises from 31.20M at 2 cores to 137.18M at 4 cores and 158M at 8 cores.

This increase highlights the limitations of the MI protocol. Because it lacks additional coherence states, the directory cannot support read sharing without transferring ownership through a GETX event. As the number of cores increases, the probability that a block is already held by another cache also increases, which leads to more ownership transfers and higher GETX traffic.

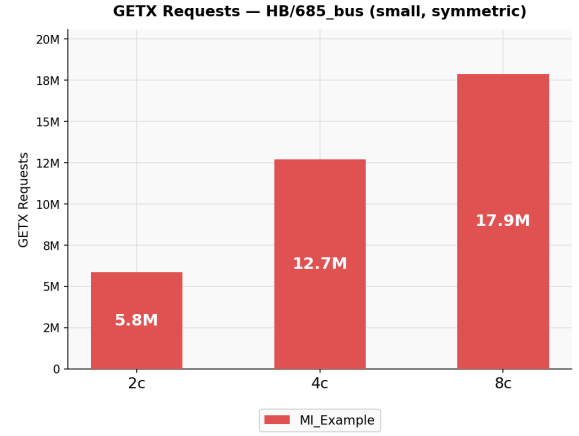


Fig. 8. GETX activity for MI_Example on HB/685_bus across 2, 4, and 8 cores.

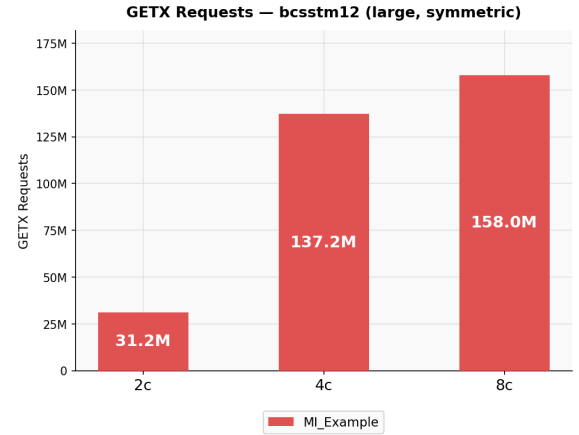


Fig. 9. GETX activity for MI_Example on HB/bcsstm12 across 2, 4, and 8 cores.

3) *FWD_GETX*: FWD_GETX traffic represents forwarded ownership requests sent from the directory to a cache that currently holds a block in the Modified state. When another core requests the same block through a GETX message, the directory forwards the request to the cache that owns the block so that it can supply the updated data and relinquish ownership. Since the MI protocol allows only one valid copy of a block at a time, these forwarded requests occur whenever ownership must move between caches.

Across all workloads, FWD_GETX activity increases as the number of cores increases. For the HB/685_bus workload, as shown in Fig. 10, FWD_GETX requests increase from 5.38M at 2 cores to 12.27M at 4 cores and 17.34M at 8 cores. A similar trend appears in the HB/bcsstm12 workload, where, as shown in Fig. 11, FWD_GETX traffic rises from 28.68M at 2 cores to 134.99M at 4 cores and 156.12M at 8 cores.

This increase reflects frequent ownership transfers between caches. Since MI allows only a single Modified copy of each cache block and has no additional states, any access by another core requires the current owner to forward the data and give up ownership. As the number of cores increases, the likelihood that another cache already holds the block also increases, which leads to more FWD_GETX events and higher

forwarded-ownership traffic.

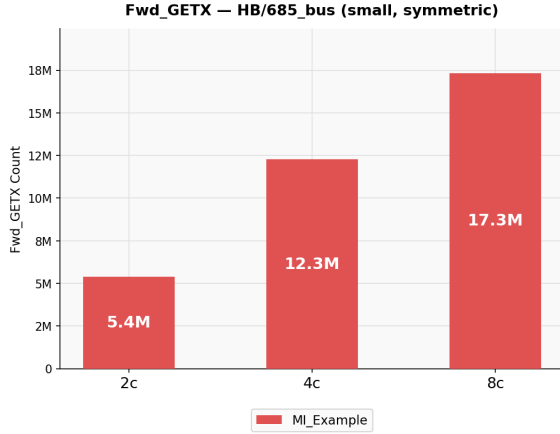


Fig. 10. Fwd_GETX activity for MI_Example on HB/685_bus across 2, 4, and 8 cores.

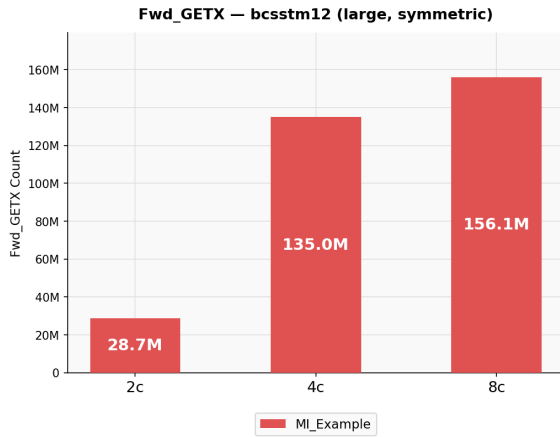


Fig. 11. Fwd_GETX activity for MI_Example on HB/bcsstm12 across 2, 4, and 8 cores.

B. MESI_Two_Level

1) *Invalidation-Related Activity*: We first examine invalidation-related coherence activity between the MI_Example and MESI_Two_Level protocols. Since MI_Example does not provide a dedicated invalidation counter, we use PUTX events as a proxy for invalidation-related behavior in MI_Example. As shown in Fig. 12, on HB/685_bus at 2 cores, MI_Example records 370.7K invalidation-equivalent events, whereas MESI_Two_Level records only 61.6K, which is approximately 6 $\times$ fewer. A similar pattern appears for the larger workload. As shown in Fig. 13, on HB/bcsstm12 at 4 cores, MI_Example generates 2.15M invalidation-equivalent events, while MESI_Two_Level records 236.4K, corresponding to roughly a 9 $\times$ reduction. This reduction suggests that MESI_Two_Level handles shared and private cache blocks more efficiently than MI_Example. In particular, the Exclusive state allows a privately held block to be written without triggering an invalidation, while the Shared state supports clean sharing without immediately forcing ownership transfer. Since the Exclusive and Shared states do not exist in MI_Example, invalidation-related

traffic remains much higher. A similar qualitative trend was also observed for HB/bp_400, where MESI_Two_Level consistently reduced invalidation-related activity relative to MI_Example.

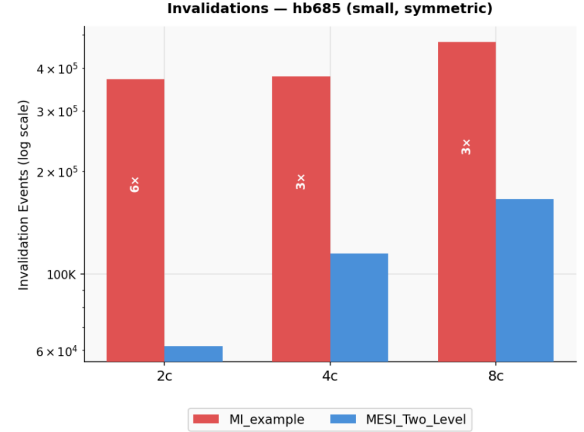


Fig. 12. Invalidation-related activity comparison between MI_Example and MESI_Two_Level for HB/685_bus.

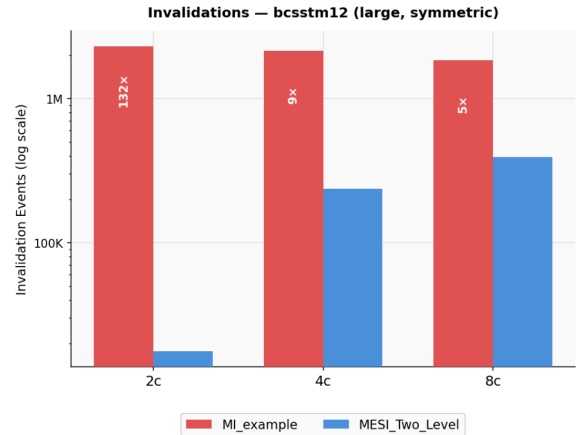


Fig. 13. Invalidation-related activity comparison between MI_Example and MESI_Two_Level for HB/bcsstm12.

2) *GETX Requests*: Next, we examine GETX traffic, which counts exclusive ownership requests issued when a core needs write permission on a cache line. As shown in Fig. 14, on HB/685_bus at 2 cores, MI_Example generates 5.85M GETX requests, while MESI_Two_Level generates only 18.7K GETX requests, which is a reduction of more than 313 $\times$. In addition, the gap becomes even larger for the larger workload. As shown in Fig. 15, on HB/bcsstm12 at 4 cores, MI_Example issues 137.2M GETX requests, compared with only 81.4K for MESI_Two_Level, corresponding to a reduction of more than 1,685 $\times$. This large divergence reflects a fundamental difference in how the two protocols handle write access. In MI_Example, ownership conflicts are resolved through a full GETX transaction regardless of whether the block is truly shared. MESI_Two_Level avoids much of this traffic through the Exclusive state. When a block has no other sharers, a later write can proceed without issuing a new GETX request,

and as workload size and sharing complexity increase, this advantage becomes even more visible. A similar reduction in GETX traffic was also observed for HB/bp_400, showing that this behavior is not limited to the small and large matrix cases.

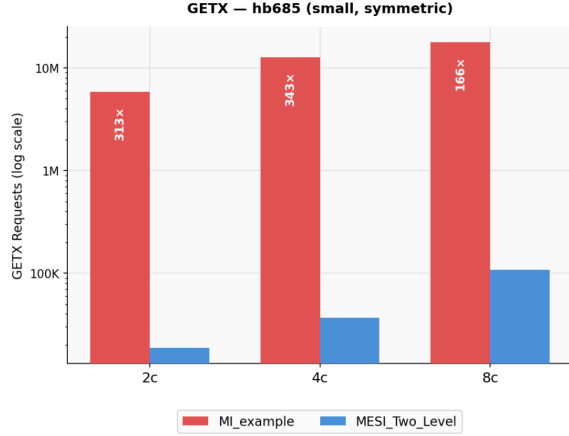


Fig. 14. GETX request comparison between MI_Example and MESI_Two_Level for HB/685_bus.

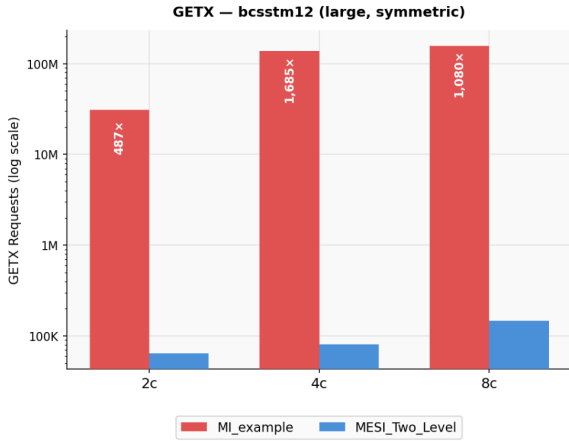


Fig. 15. GETX request comparison between MI_Example and MESI_Two_Level for HB/bcsstm12.

3) *E.Store Activity*: In order to understand the benefit of the Exclusive state in the MESI_Two_Level protocol's reductions in invalidation-related activity and GETX traffic, we examine the E.Store event, where MESI_Two_Level records silent Exclusive-to-Modified upgrades. Each E.Store event corresponds to a write that completed without issuing a GETX request and without causing invalidation activity. Because MI_Example has no Exclusive state, it has no equivalent counter. As shown in Fig. 16, for the HB/685_bus workload at 2 cores, MESI_Two_Level records 171.9K silent upgrades. For the larger workload, the effect becomes even more pronounced. As shown in Fig. 17, E.Store on HB/bcsstm12 increases from 315.7K at 2 cores to 421.4K at 4 cores. These results help explain why GETX traffic drops so sharply in MESI_Two_Level. When the protocol is able to keep a block in the Exclusive state, later writes can be handled locally

without a new ownership request. The increase in E.Store for the larger matrix suggests that bcsstm12 creates more effectively private regions as the working set is distributed across multiple caches.

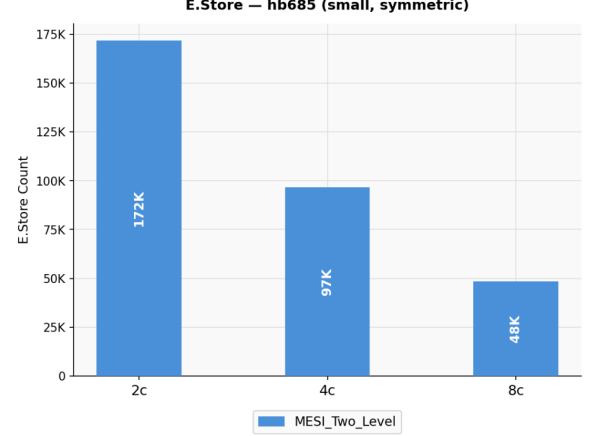


Fig. 16. E.Store activity in MESI_Two_Level for HB/685_bus.

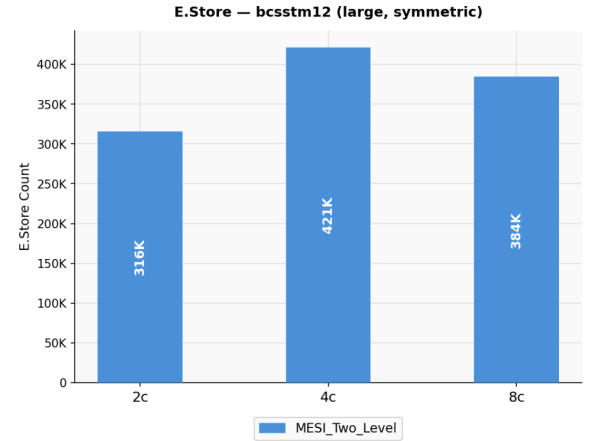


Fig. 17. E.Store activity in MESI_Two_Level for HB/bcsstm12.

4) *GAPBS BFS*: In all three SpMV workloads, MESI_Two_Level consistently outperformed MI_Example in terms of invalidation-related activity, GETX traffic, and E.Store behavior. A similar trend was also observed for GAPBS BFS. MESI_Two_Level reduced both invalidation-related activity and exclusive ownership traffic compared to MI_Example. However, the magnitude of the reduction differed from that observed in the SpMV cases because graph traversal has a different balance of shared reads, ownership changes, and synchronization behavior than SpMV.

C. MESI_Two_Level vs. MOESI_CMP_directory

Finally, we examine the improvements of MOESI_CMP_directory over the MESI_Two_Level protocol.

1) *Invalidation Activity*: As shown in Fig. 18, we continue to observe a reduction in invalidation-related activity, especially for the GAPBS BFS workload. This was especially prevalent in our GAPBS experiment, where the decrease in

invalidation-related traffic can be attributed to sharing among caches decreasing ownership requests and allowing caches to share without changing permissions. BFS results in a large amount of write contention from writing to visited node and distance lists, so if nearby nodes are located in the same cache block, writes from different cores to different elements can cause false sharing misses and frequent ownership transfers as a result. MOESI allows dirty values to be shared, allowing for less invalidations and coherence messages. In terms of the protocols, the ownership state aims to prevent additional writebacks and optimize reads between caches when sharing. It's possible that MOESI's optimized sharing is helping to keep cache blocks within their local cache working sets for a longer period of time, leading to fewer invalidations compared to MESI.

Invalidation-related activity for the GAPBS BFS workload remains lower in MOESI CMP directory than in MESI Two Level across all tested core counts, although both protocols still show increasing coherence pressure as core count rises. This is because higher levels of parallelism increase contention among cores accessing shared data. The probability that multiple cores update data on the same cache blocks (write contention) also increases with more cores. When a core attempts to write to a cache block that is already shared by other cores, it must issue a GETX request and simultaneously invalidate other cache copies to gain exclusive ownership. As a result, cache blocks are transferred between cores more frequently, producing more coherence invalidations.

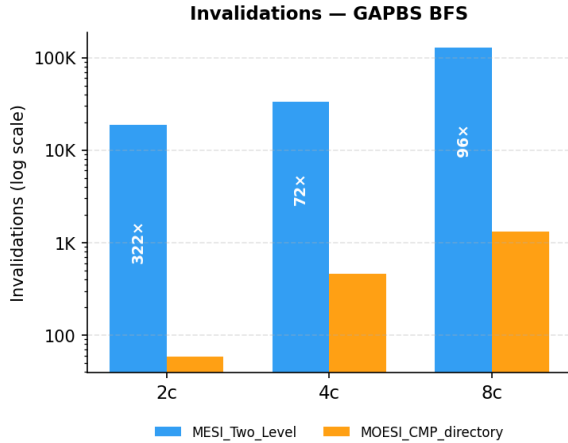


Fig. 18. Invalidation-related activity comparison for GAPBS BFS between MESI_Two_Level and MOESI_CMP_directory.

2) *FWD_GETS and GETS Activity*: Another trend observed across all workloads was a decrease in GETS messages, but an increase in FWD_GETS. As mentioned before, FWD_GETS messages are a direct indicator of an owning cache forwarding its value to caches requesting reads of the block being owned, highlighting the usage of the Owned state to share its up-to-date value with other readers without going to main memory. As shown in Fig. 19, we observed the largest proportional increase in FWD_GETS for the small symmetric matrix, a 20× increase from 8,393 to 163,143 messages, likely due to the small matrix size fitting more easily in the cache

and leading to more overlapping reads. (reading the same blocks repeatedly) However, FWD_GETS were consistently over 3 times higher for all matrix and BFS workloads. For symmetric matrices, increasing the size tends to produce more FWD_GETS activity, so the symmetry could increase the opportunity for shared reads to forward owned data to other caches. The same is true for increasing the number of cores, which increases forwarding activity.

However, for unsymmetric matrices, FWD_GETS messages are the lowest at 2 cores, peak at 4 cores to about 207,000, then decrease for 8 cores to 149,000 messages for the MOESI_CMP_directory protocol. With 2 cores, the matrix is split into large partitions that limit sharing due to having less cores to forward data with. At 4 cores, the likelihood of sharing increases, since the partitions become smaller and overlapped access increases. For 8 cores, the working sets for each become too small, and each core operates on more localized portions of the matrix. This results in less overall FWD_GETS sharing due to reduced overlap in computations. For this particular unsymmetric workload, the optimal number of cores and partition size is around 4 cores to optimize working set size and core forwarding potential, as shown in figure 20.

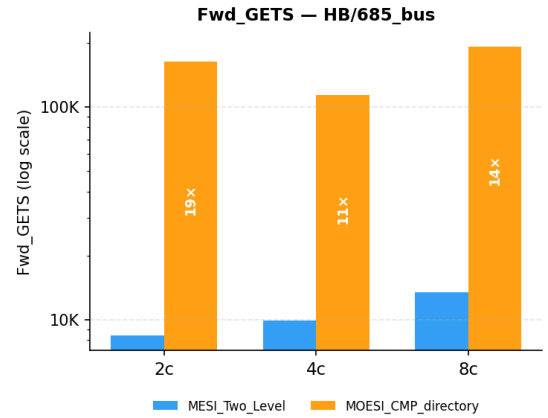


Fig. 19. FWD_GETS comparison for HB/685_bus between MESI_Two_Level and MOESI_CMP_directory.

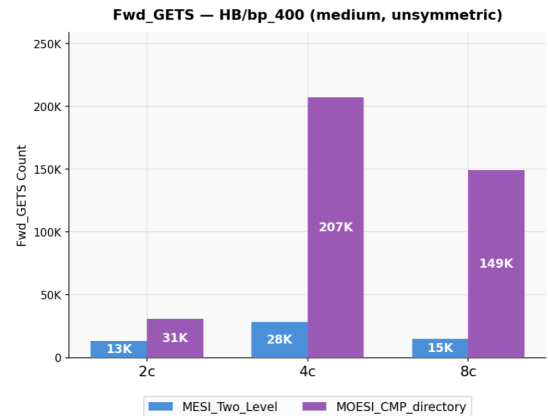


Fig. 20. FWD_GETS comparison for BP400 unsymmetric matrix between MESI_Two_Level and MOESI_CMP_directory, non log-scale. For 2 cores, the unsymmetric matrix exhibits very little sharing, sharply increases at 4 cores, then decreases slightly for 8.

As shown in Fig. 21, MOESI generates much less GETS traffic than MESI for the graph BFS workload. The number of GETS for MESI increases from about 91,000 to 136,500 from 2 to 8 cores, but remains at around 19,000 regardless of core count for MOESI. This can be attributed to having fewer read misses in MOESI. MESI forces extra memory reads in order to read values from main memory rather than from owning caches. No main memory accesses are required when a core reads owned data, which may be reducing the number of GETS. When the number of cores increases for the graph workload, BFS becomes more concurrent, resulting in more cores reading the same neighbors. This creates more writebacks to shared memory locations, increasing the number of GETS.

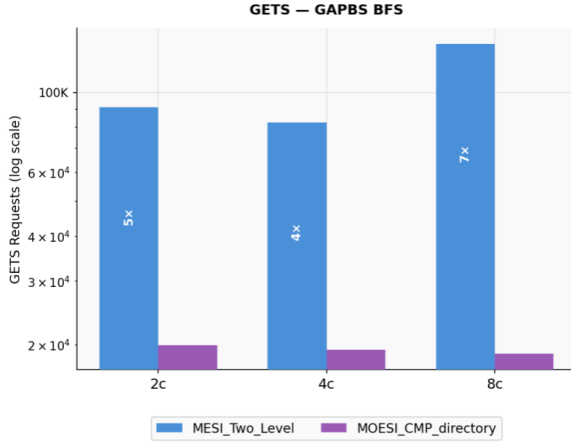


Fig. 21. GETS comparison for GAPBS between MESI and MOESI.

3) *GETX Activity*: We also observed fewer exclusive ownership requests (GETX), although a non-negligible amount of GETX traffic was present in matrix multiplication. This is possibly due to the size of the output data not being able to fit entirely in the local cache. This causes blocks to be evicted then later required with GETX on later writes. In both symmetric and asymmetric matrices, the number of GETX requests is relatively stable with small increases as the number of cores grows. (see Figs. 22 and 23)

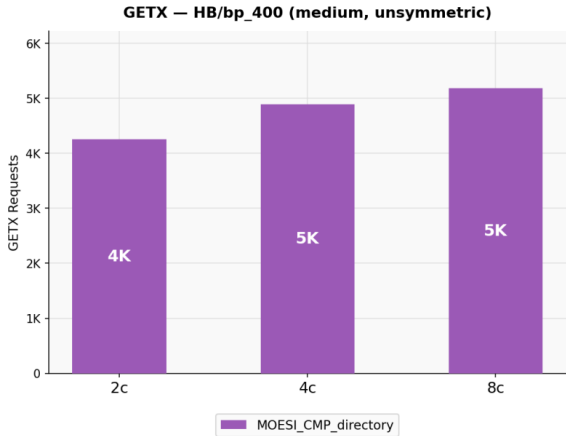


Fig. 22. GETX comparison for BP400 unsymmetric matrix for 2,4 and 8 cores for MOESI.

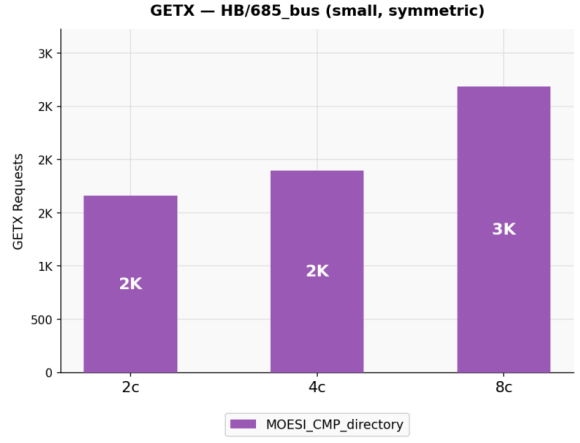


Fig. 23. GETX comparison for HB685 symmetric matrix for 2,4 and 8 cores for MOESI..

V. DISCUSSION

As evidenced by the results of the experiments, we can see that the overall performance improvements provided by adding more states in which our memory blocks can use heavily adds flexibility with the way in which our coherency protocol adapts to memory access patterns to handle workloads of any kind. However, with the spike in FWD_GETS requests for both studied workloads, one could assume that perhaps some of these forward requests may be caused by true sharing misses due to multiple cores perhaps needing to request a specific memory block to write to it. In this situation, multiple cores would have to send their own forwarding requests waiting for this mutual memory block to finally be provided to the core after previous cores are done. In this scenario, there would be a large amount of requests to this specific block that would require passing these 128 byte memory blocks back and forth to each of the requesting cores. It's easy to see how this could heavily congest valuable interconnect bandwidth. Although there is no metric in Gem5 that proves such workloads would lead to a concerning level of true sharing misses, one test that can be done is to change the memory block size to just one word and analyze the rise or drop in the number of forwarding. However, we could not fit this extra experiment within the time frame of the project, and will have to mark it off as a future experiment that we can use to explore.

Assuming the true sharing is indeed a problem, a potential way to solve this back and forth between requesting cores is to introduce a "Delegated" state (D) to memory blocks that are frequently requested for by cores within a specific time frame from the first request. The idea is that when a memory block is in the D state, the core that holds the D state memory block will be the sole writer to this memory block until traffic to the memory block eases. The idea of a sole writer, is that other cores that want to write to the memory block will just send their lightweight write instructions to the sole writer. This in theory would be a cheaper operation than sending the large memory block over to the requesting core, and thus is beneficial overall for frequently written to blocks. However, this state will require more complex hardware in place in order to allow such a protocol to work. There would

need to be a dedicated counter that of how many times a memory block is being accessed within a time frame, in order to decide whether or not a block should be put in the D state or not. There are also other factors to consider, but for now we will not consider potential problems with that would need to be addressed to implement this protocol as this idea is still in it's infancy. But we leave this idea on the table for future experimentation and design.

VI. FUTURE WORK

In order to boost our overall confidence in our confidence in the analysis we presented in this paper, there are several experiments that have yet to be conducted that would potentially unveil underlying trends in standard coherency protocols in ML workloads. One such experiment was mentioned earlier, but Gem5 lacks metric that could indicate the amount of true sharing and false sharing misses when exclusive access requests are made to modified blocks. A potential workaround this shortcoming of the Gem5 simulator is that we can run a separate experiment in which we measure the effect on the message traffic when we reduce the memory block size to a single word. This way, we now for certain when a specific value or node is accessed, and see how much true sharing misses were at play during the execution time of these ML workloads. This would provide us ample evidence into the underlying problems with these standard protocols, and lead us in a direction for improving upon these standard protocols.

It should also be noted that for our Sparse Matrix workload, we decided on implementing our own workload program in order to benchmark SpMV workloads. However, after analyzing some of the results from our experiments, we had some discrepancies in our data that defied our intuition of how coherency protocols work. For example, we noticed a slight drop in the invalidation messages that were sent as the core counts increased in the MI_EXAMPLE protocol. The only attributable way this result may be possible was perhaps because we provided a custom implementation. So for a future work, we would like to instead use a more well documented and standardized benchmark for sparse matrix operations such as the SparseSuite repository. This would provide accurate data that would most likely eliminate in the discrepancies in the data that may have been attributed to incorrect setup in our custom implementation.

VII. CONCLUSION

The rapid shift in machine learning toward irregular, highly sparse workloads, such as Sparse Matrix-Vector Multiplication (SpMV) and Graph algorithms like BFS, has exposed critical inefficiencies in traditional multicore memory subsystems. In this paper, we systematically evaluated the impact of these chaotic, data-dependent memory access patterns on directory-based cache coherency protocols using the Gem5 Ruby simulator. By isolating the behavior of the MI, MESI, and MOESI protocols under identical architectural parameters, we quantified how incremental additions to coherency state machines mitigate network congestion. Our results demonstrate that

the strict two-state limitations of the MI protocol result in severe interconnect bottlenecking, characterized by excessive invalidation traffic and GETX requests. The introduction of the Exclusive and Shared states in MESI provided substantial relief by enabling clean sharing and silent upgrades, which reduced GETX requests by orders of magnitude. Furthermore, the MOESI protocol demonstrated the distinct advantages of the Owned state for ML workloads. By permitting "dirty sharing," MOESI drastically reduced expensive main memory accesses (GETS messages) and invalidations, instead serving read requests directly from owning caches via FWD_GETS messages.

However, the significant spike in forward messages observed under MOESI underpins a lingering challenge: the heavy contention and "ping-ponging" of cache lines driven by both false and true sharing among concurrent cores. To address the bandwidth tax of transferring highly contended 128-byte cache blocks back and forth, we proposed the conceptual "Delegated" (D) state. By designating a single core as the sole writer and allowing other cores to dispatch lightweight, atomic write instructions, future coherency protocols could bypass the overhead of standard ownership transfers for heavily shared data structures.

Ultimately, as ML models continue to scale in sparsity and size, evolving cache coherency protocols to intelligently handle irregular memory accesses will be essential to overcoming the memory wall and sustaining parallel performance.

REFERENCES

- [1] "SuiteSparse Matrix Collection," *Texas A&M University*. [Online]. Available: <https://sparse.tamu.edu/>. Accessed: Mar. 11, 2026.
- [2] "Matrix Market I/O example program." [Online]. Available: https://math.nist.gov/MatrixMarket/mmio/c/example_read.c. Accessed: Mar. 10, 2026.
- [3] sbeamer, "GitHub - sbeamer/gapbs: GAP Benchmark Suite," *GitHub*. [Online]. Available: <https://github.com/sbeamer/gapbs>. Accessed: Mar. 11, 2026.
- [4] gem5, "MESI Two Level," *gem5 Documentation*. [Online]. Available: https://www.gem5.org/documentation/general_docs/ruby/MESI_Two_Level/. Accessed: Mar. 11, 2026.
- [5] gem5, "MI Example," *gem5 Documentation*. [Online]. Available: https://www.gem5.org/documentation/general_docs/ruby/MI_example/. Accessed: Mar. 11, 2026.
- [6] gem5, "MOESI CMP Directory," *gem5 Documentation*. [Online]. Available: https://www.gem5.org/documentation/general_docs/ruby/MOESI_CMP_directory/. Accessed: Mar. 11, 2026.
- [7] gem5, "gem5 bootcamp 2024: Computer Architecture Simulation," *YouTube*. [Online]. Available: <https://www.youtube.com/watch?v=6ThcR6jKwkc>. Accessed: Mar. 12, 2026.